VISIUM

Pioneer a Bright Future

DARE TO BUILD YOUR DATA-DRIVEN FUTURE

Agentic fights 

By Alejandro Medrano & Carlos Martínez

Agentic workshop at UPV, in Valencia!

Thanks to María!

Agentic workshop at UPV, in Valencia!

CONTENTS

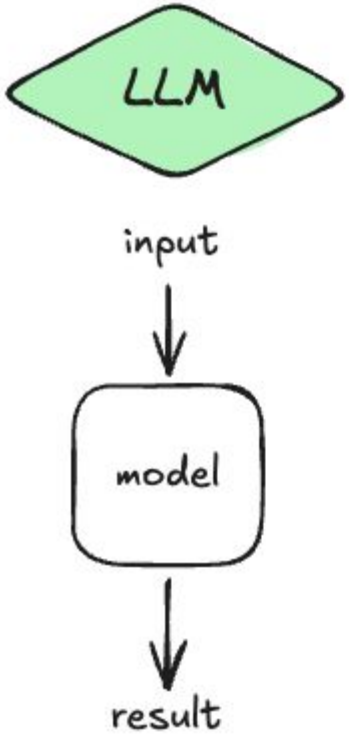
CONTENTS

What is an agent?

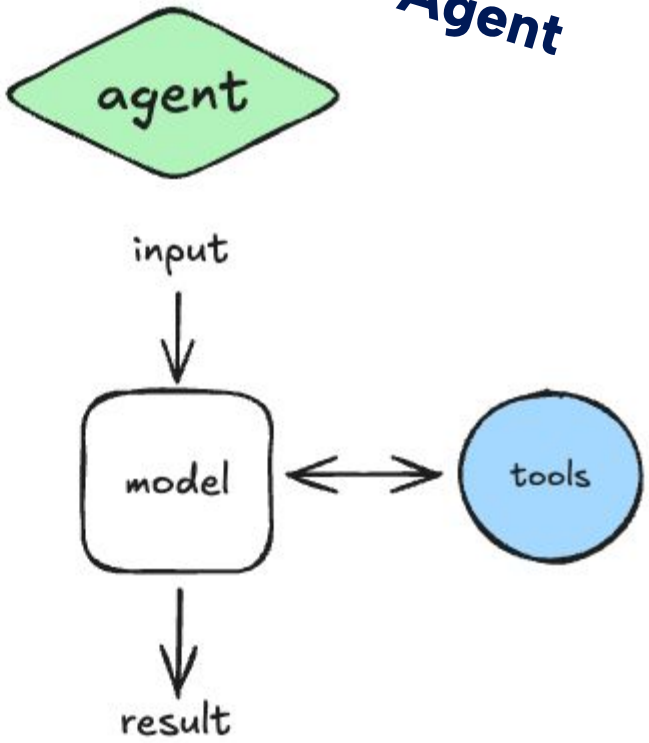
- An agent is a system that **perceives its environment, makes decisions and takes actions autonomously**.

- Instead of programming a fixed control flow, sometimes we want LLM systems that can choose their own control flow to solve more complex problems:
  - an LLM can decide between two potential routes
  - an LLM can decide which of many tools to call
  - an LLM can decide if the generated response is sufficient or if more work is needed

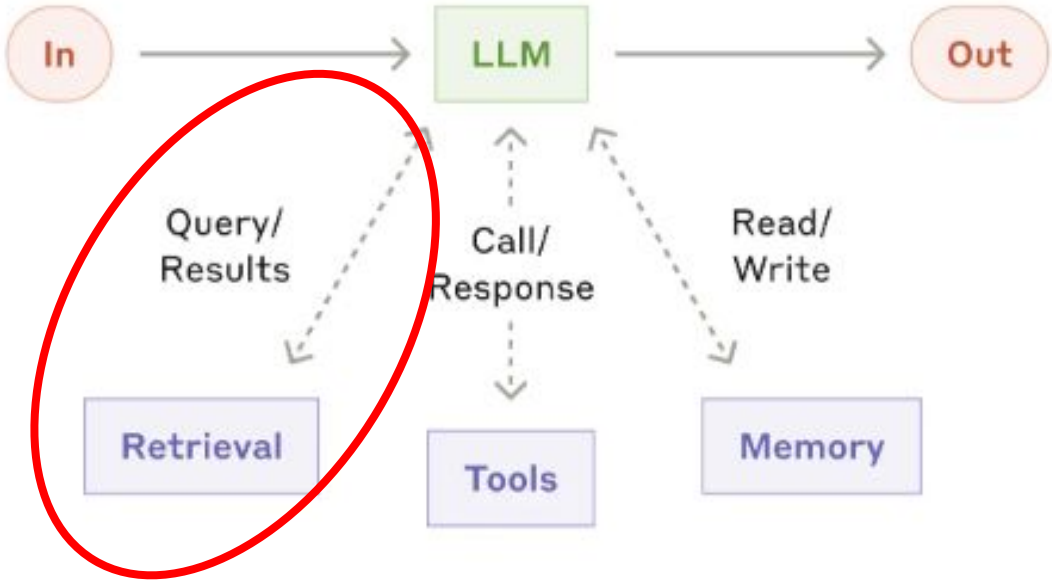
Plain LLM



Agent



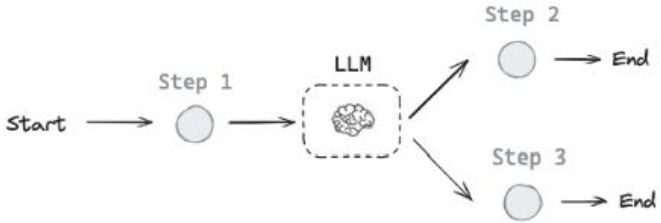
The augmented LLM



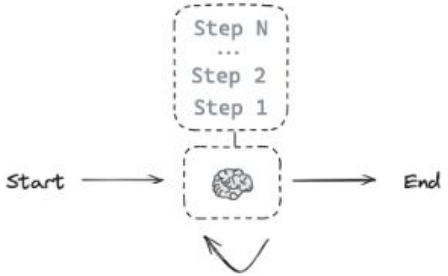
A simple RAG is already considered an agent!

Different levels of autonomy

“Workflows”



Autonomous agent



Benefits of using agents

2025 is the year of the agents. Agent-based approaches are reshaping how we build intelligent systems, because they are becoming more practical, powerful, and scalable.

- **Autonomy**: agents can plan, make decisions, and act independently, freeing developers from writing rigid, step-by-step logic. Moreover, agents can be topic experts, improving its performance
- **Adaptability and reasoning**: agents leverage powerful LLM reasoning to adapt in real time and reevaluate plans
- **Multi-agent collaboration**: systems like crew.ai and Autogen enable agents to collaborate, delegating tasks and coordinating like a human team
- **Scaling through modularity**: their design allows for composability, plugging in new capabilities (like retrieval, vision, or APIs) without retraining (retrieval, functions, APIs… MCP standardization)

Creating an agent from scratch: ReAct

ReAct is the foundational framework for LLM-based agents. It provides a simple but powerful way to structure agent behavior using LLM outputs.
It blends two essential capabilities:

- Re: **Reasoning** (what to do next)
- Act: **Acting** (taking an action, like calling a tool)

This framework works because:

- It keeps the LLM grounded by seeing results of its actions
- It supports tool use, multi-step problem solving, and self-correction
- It's easy to implement with just a loop and a few function calls

This forms the basis of many modern agents (LangChain agents, OpenAI Function Calling, Autogen, etc.)

Creating an agent from scratch: ReAct

This loop enables the agent to think step-by-step and use tools or external APIs to gather new information before continuing.

One example: ***“How much do I have to pay if I want 5 samples of this item?”***

1. Thought: internal reasoning → *“I should look up the current price of the item”*
2. Action: external operation → *Search["item price"]*
3. Observation: the result of that action → *The price is $19.99*
4. (Repeat): think again, or finalize the answer → *“Now that I know the price, I can answer”*
5. Final answer: **output the result**

It’s coding time!

Let’s create an agent from scratch

CONTENTS

Why LangGraph?

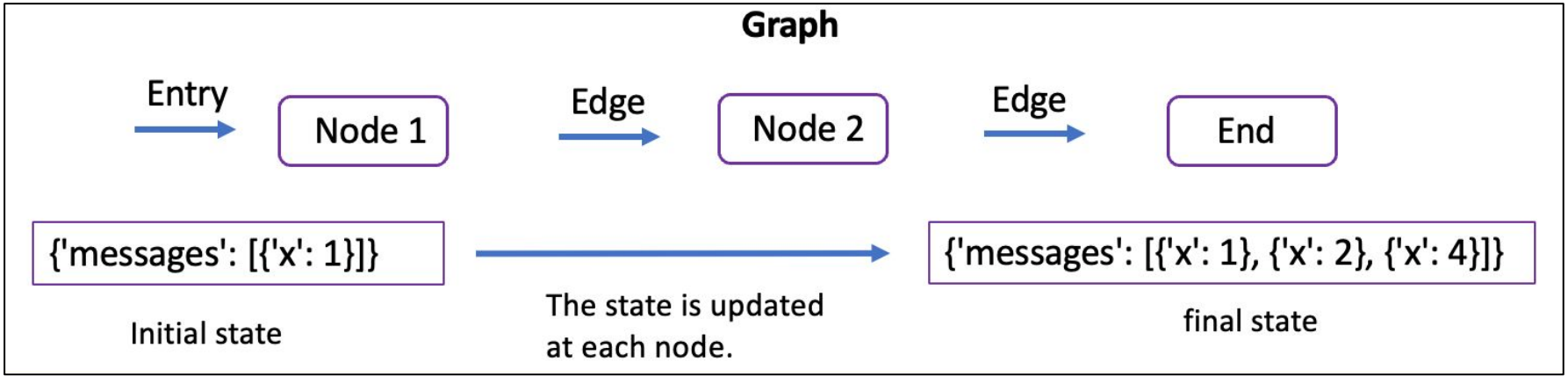
LangChain does not have a good reputation among engineers… But LangGraph does!

We worked on this only 5 months ago, but there were no other solid alternatives

LangGraph 🔗

- Extension of LangChain to **define workflows as graphs**
  - it builds on top of LangChain's useful message system
- Nicely structured and easy to learn → **great control over the whole process**
- **Production ready**
- Under heavy development and **incorporating great additions (like memory)**

Why LangGraph?



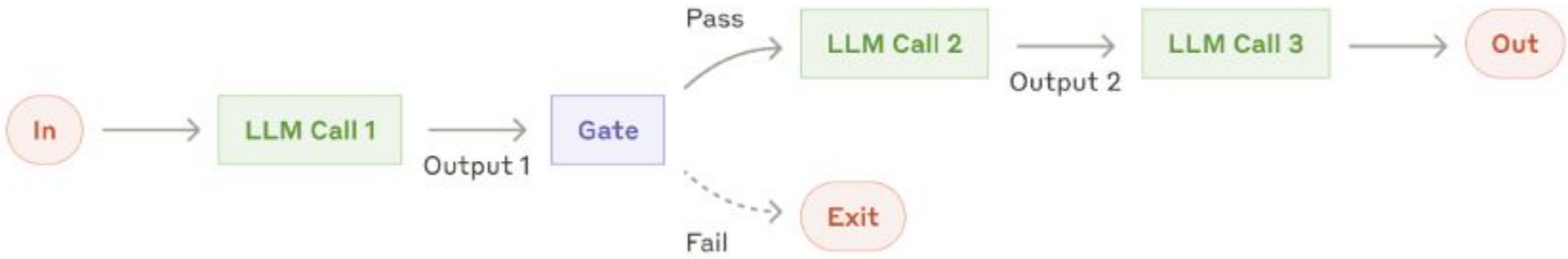
Key concepts: based on graph theory

- **Node**: functions where code is executed
- **Edge**: relations between nodes
  - direct
  - conditional
- **State**: a dictionary that is updated in every node

- **Persistence**: an extension of the state to have memory between sessions

Simple agent architectures

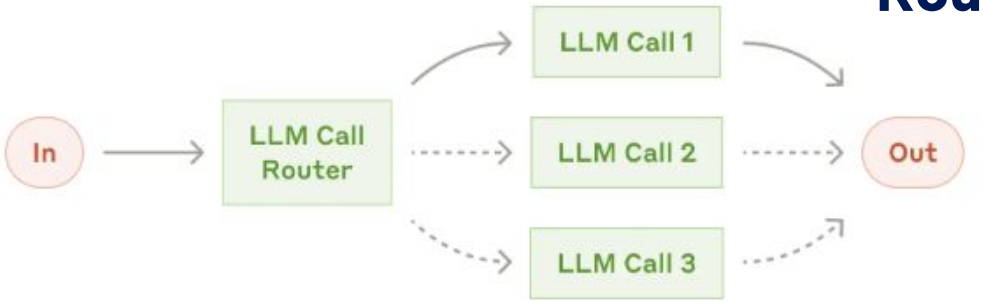
Prompt chaining



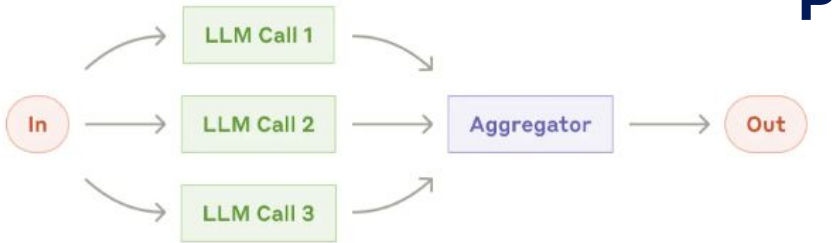
Let’s see how is this done with LangGraph…

Simple agent architectures

Routing



Parallelization

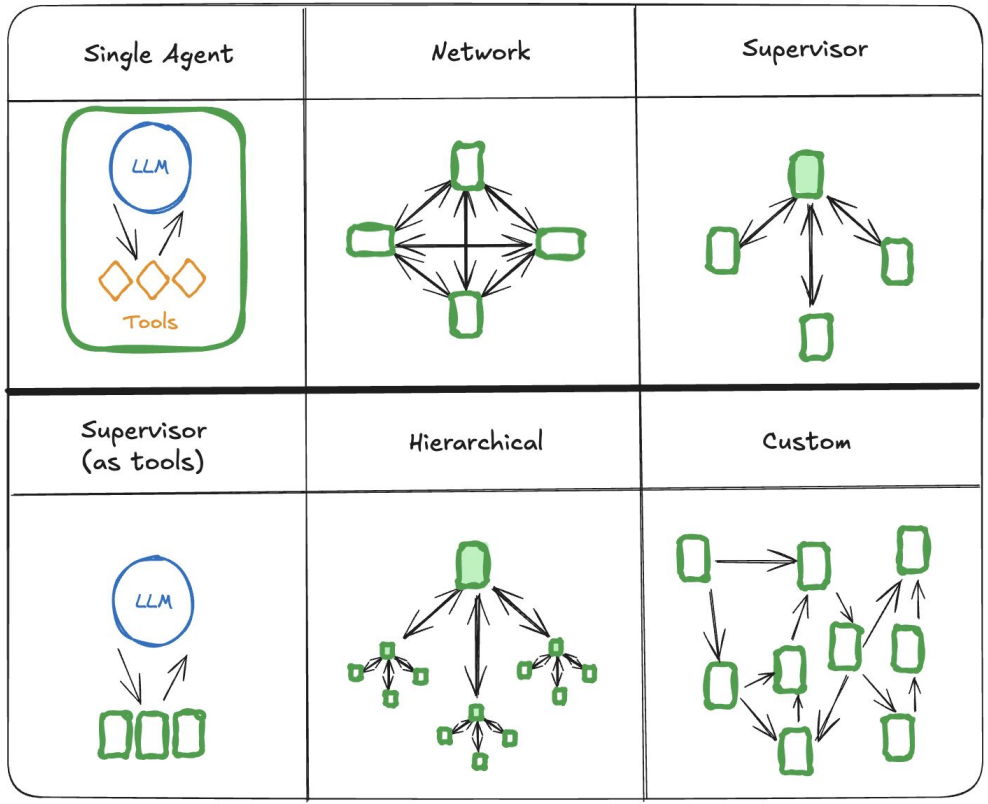


Simple agent architectures

Orchestrator – Workers

Evaluator – Optimizer

Multi-agent architectures



CONTENTS

Agentic Fights! 🦜🔗

Main ideas:

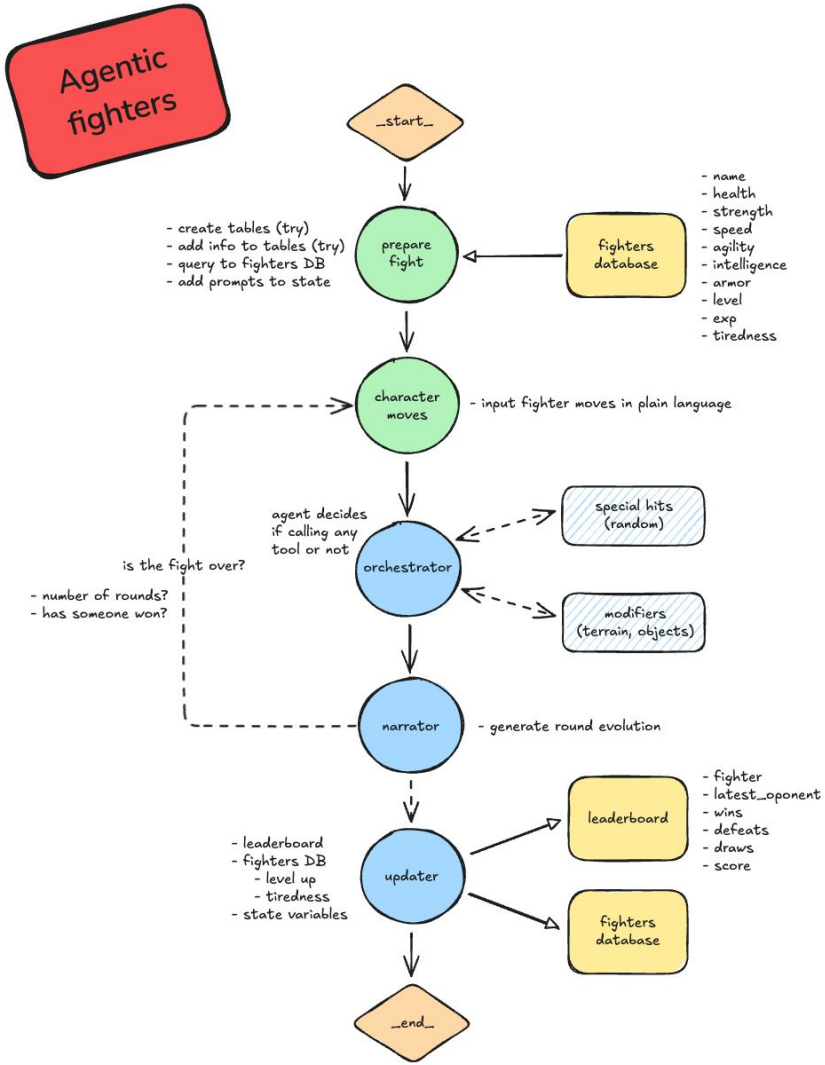
- To develop a role-playing fighting game using the capabilities of genAI, in an agentic way
- Include memory and some extra features, to somehow mimic an actual game

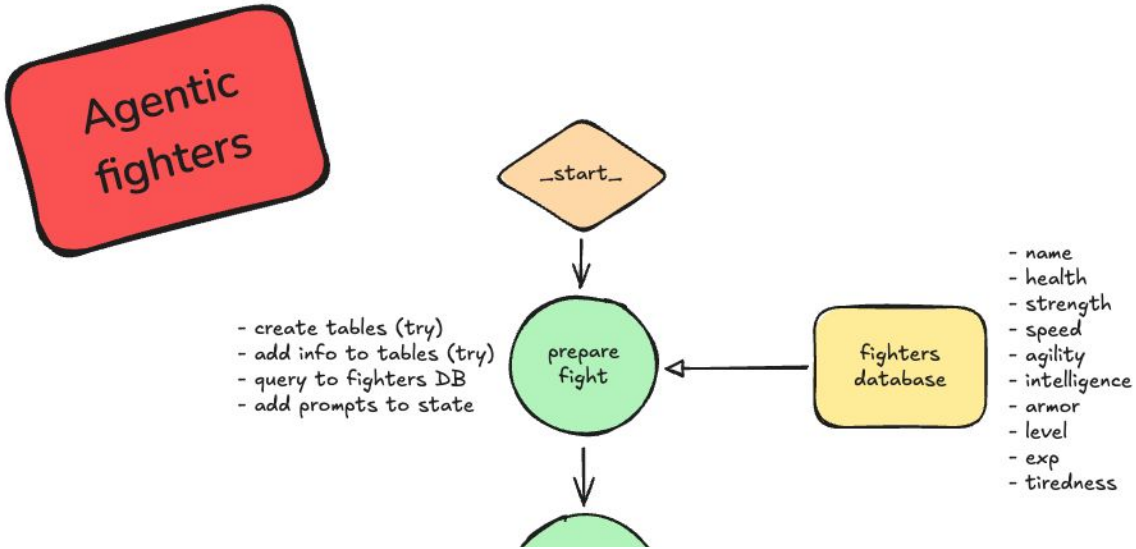
- Serve as a simple basis to get to replace gamemasters in a role-playing game

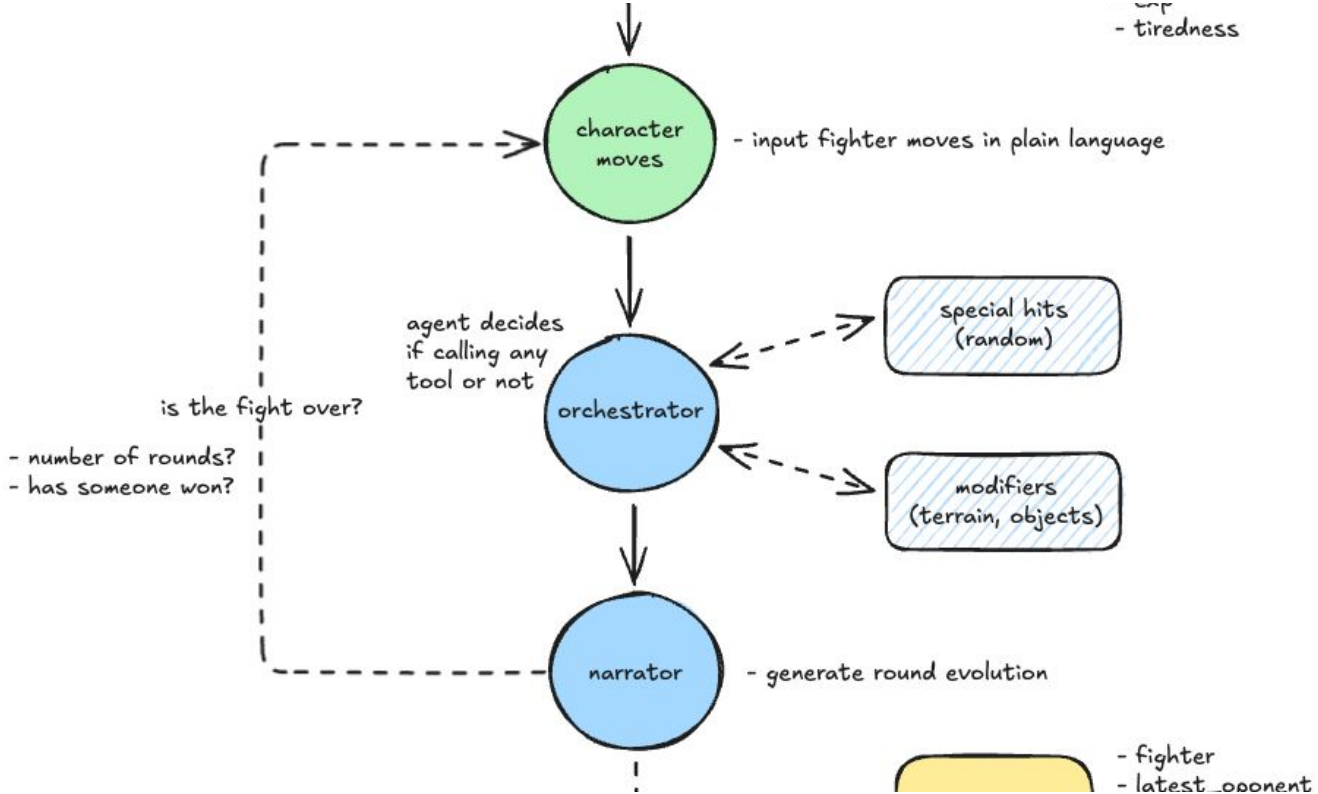
Agentic Fights! 🦜🔗

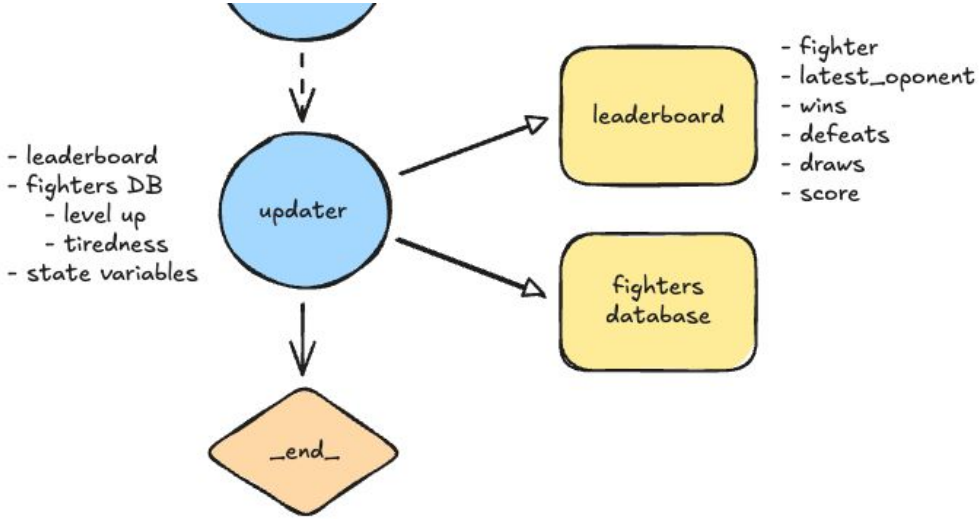
How does it work?

- Everything is customizable via prompts
- Some nodes are static, meaning they execute pure code to generate the database, read the characters info, etc.
- Other nodes are generative, using the capabilities of the LLMs to generate the actual fight
- The relations between nodes are direct or conditional, creating the whole graph cycle. They define the logic of the execution, based on the generative results
- There’s in memory persistence, so that the executions are continuous


Agentic fighters
start
prepare fight
- create tables (try)
- add info to tables (try)
- query to fighters DB
- add prompts to state
fighters database
- name
- health
- strength
- speed
- agility
- intelligence
- armor
- level
- exp
- tiredness
character moves
- input fighter moves in plain language
agent decides if calling any tool or not
orchestrator
special hits (random)
modifiers (terrain, objects)
is the fight over?
- number of rounds?
- has someone won?
narrator
- generate round evolution
leaderboard
- fighter
- latest_oponent
- wins
- defeats
- draws
- score
- leaderboard
- fighters DB
- level up
- tiredness
- state variables
updater
fighters database
end


Agentic fighters
start
- create tables (try)
- add info to tables (try)
- query to fighters DB
- add prompts to state
prepare fight
fighters database
- name
- health
- strength
- speed
- agility
- intelligence
- armor
- level
- exp
- tiredness


- tiredness
character moves
- input fighter moves in plain language
agent decides if calling any tool or not
special hits (random)
orchestrator
is the fight over?
- number of rounds?
- has someone won?
modifiers (terrain, objects)
narrator
- generate round evolution
- fighter


- leaderboard
- fighters DB
- level up
- tiredness
- state variables
updater
leaderboard
- fighter
- latest_oponent
- wins
- defeats
- draws
- score
fighters database
end

Let’s see it in action!

Who will win?

Make your bets!

CONTENTS

Conclusions

- **Agents are here to stay**, but there's of research to be done
  For now, we have:
  - More powerful assistants
  - Giant leap for automatization

- **Workflow vs Pure agents**
  - Workflow: "predefined" steps
  - Pure agent: dynamic, autonomous, decision making and execution

- **Scalable Intelligence:** multi-agent systems can collaborate, delegate, and specialize like real teams

- **Expanding ecosystem**
  - *LangGraph, CrewAI, Haystack, OpenAI agents (Swarm some months ago), Autogen, …*

Conclusions

LangGraph is not perfect

- Clumsy sometimes
- Difficult to integrate with a Streamlit webapp, because of the way it's executed
- Some actions are not very well implemented (Human interruptions)

Things to be done

- Try a different framework, or even no framework at all
- Include recent additions like MCP to enhance the agents
- Make the multiplier tool work properly…

Do we really need a framework?

- Sometimes, using a framework is just an overkill
- Frameworks or libraries = bound to how other person thinks
- With **structured outputs**, one can create their own processes and flows

Questions?

VISIUM

Pioneer a Bright Future

DARE TO BUILD YOUR DATA-DRIVEN FUTURE

Agentic fights 

By Alejandro Medrano & Carlos Martínez